

The Developer's Cookbook

Recipes for Better Code

Recipe: The Perfect Function

Ingredients

- One clear purpose
- Descriptive name
- 3-5 parameters (max)
- Single return type
- A pinch of error handling

Instructions

1. Start with the name. If you can't name it clearly, it's doing too much.

2. List your parameters. If you need more than 5, consider a configuration object.
3. Write the happy path first. Make it work, then make it robust.
4. Add error handling. Fail fast, fail clearly.
5. Test with edge cases. Empty inputs, null values, boundary conditions.
6. Refactor until it reads like prose.

Serving Size

One function serves one purpose. If it serves multiple purposes, split it.

Recipe: The Maintainable Codebase

Ingredients

- Consistent style guide
- Clear directory structure
- Comprehensive tests
- Living documentation
- Regular refactoring

Instructions

1. Establish conventions early. Consistency is more important than perfection.
2. Organize by feature, not by type. Group related code together.
3. Write tests as you go. Tests are your safety net.
4. Keep documentation close to code. README files, docstrings, comments.
5. Schedule refactoring time. Technical debt compounds like interest.

Cooking Time

This is a slow-cook recipe. It takes months, even years. But the result is worth it.

Recipe: The Debugging Session

Ingredients

- Reproducible bug
- Fresh perspective

- Good logging
- Rubber duck (optional but recommended)

Instructions

1. Reproduce the bug consistently. If you can't reproduce it, you can't fix it.
2. Add logging. See what's actually happening, not what you think is happening.
3. Explain the bug to your rubber duck. Often, the explanation reveals the solution.
4. Make one change at a time. Isolate variables.
5. Test your fix. Does it solve the problem? Does it break anything else?
6. Document the fix. Why did it break? How did you fix it?

Pro Tip

The bug is usually in the last thing you changed. But not always.

Recipe: The Code Review

Ingredients

- Open mind
- Constructive feedback
- Empathy
- Learning attitude

Instructions

1. Read the code, not the person. Critique the code, not the coder.
2. Ask questions, don't make demands. "Have you considered X?" not "You should do X."
3. Point out what's good. Positive feedback is just as important as constructive criticism.
4. Suggest improvements, don't just point out problems.
5. Learn from the review. Every review is a learning opportunity for both parties.

Serving Suggestion

Serve with humility and gratitude. Code reviews make everyone better.